

Using PulseTrack

Every screen, what happens when you click, and the rules the app will not break — all three tiers, end to end.

WHO SIGNS IN	COVERS	READ IT AS	LIVE
Clinicians only. Patients never get an account.	All six Tier 1 areas, all four Tier 2 requirements, and Tier 3.	A walkthrough. Each section is one screen.	pulse-track-joe.vercel.app

ORIENTATION

Five screens, one register, one shared spine

PulseTrack is a clinician tool. Everything it does hangs off a **patient register**, and every record — a lab result, a questionnaire score, a row on the national platform — is attached to a patient by their **MRN**. Learn the register and the rest follows.

01 Clinic The overview. Four figures for the whole practice, the spread of risk across the register, and what has been imported lately.	02 Patients The register itself. Search, add, open a record. Each row already carries the patient's latest risk band and most recent lab result, so triage happens here rather than one record at a time.
03 Lab import Upload a CSV of lab results. Every row is judged individually and the outcome is reported line by line.	04 National platform Tier 2. Push our records to the shared FHIR server, pull twelve months of history back, and see what was refused.

05

The patient page

One person: who they are, four charts, and every assessment and lab result as a table. This is the screen you open with the patient in the room.

06

The questionnaire

The only public page. The patient opens it from a link in their email — no account, no password.

Navigation is a fixed rail on the left from a laptop width up, and a horizontal bar on a phone. The signed-in clinician and **Sign out** sit at the foot of the rail.

STEP 1

Signing in

Open the site and you land on `/login`. There is no sign-up: clinician accounts are seeded, and the credentials for the demo account are in the README.

1.1 Enter the email and password, press Sign in

- On success you are sent to **Clinic**, the dashboard. A session cookie is issued; it is a JWT, so nothing about the session is stored server-side.
- On failure the message is **“Invalid email or password”** — the same words for an unknown address as for a wrong password. Telling the two apart would turn the form into a way of discovering who has an account.
- What you typed stays in the fields. A failed submission that also clears the form makes the user retype something that may well have been right.

1.2 Try to skip it

Paste `/dashboard` or any patient URL while signed out and you are redirected to the login page. Authorisation is checked in **three** independent places, because any one of them can be misconfigured:

- **At the edge** — the proxy checks the session cookie before the page is ever rendered.
- **In the page** — every dashboard screen re-checks the session server-side when it renders.
- **In the mutation** — every action that writes calls `requireClinician()`, and every API route calls `requireClinicianApi()`. This is the boundary that actually holds; the first two are conveniences.

Hidden buttons are not a security model. A signed-out `POST` straight at `/api/labs/upload` is refused by the third layer, not by the absence of a button.

Adding a patient

Patients → Add patient. Six fields, four of them required. The rules are deliberately strict where being wrong is dangerous and permissive where being unfamiliar is not.

FIELD	REQUIRED	RULE, AND WHY
Full name	Yes	Up to 200 characters. Stored as typed.
MRN	Yes	Letters, numbers, spaces, hyphens and underscores. Uppercased and trimmed on the way in , so <code>mrn-1001</code> and <code>MRN-1001</code> cannot become two patients. Unique across the register.
Date of birth	Yes	<code>YYYY-MM-DD</code> . Must be a date that exists (<code>2026-02-30</code> is refused rather than rolled forward), not in the future, and within the last 130 years.
Sex	Yes	Chosen from a list.
Email	No	Lower-cased. Needed to send an assessment — without it the Send button reports that the patient has no address rather than failing silently.
Phone	No	Digits with <code>+ () - .</code> and spaces. Permissive on purpose: a rule that rejects <code>+961 3 111 001</code> is wrong for most of the world.

What happens when you save

- The input is validated on the server, not only in the browser. Field errors come back attached to their field, **with what you typed still in it**.
- A duplicate MRN is caught as a **field error on the MRN box** — “A patient with this MRN already exists” — not as a crash. This was a real defect once: the database constraint fired and the page died with a 500.
- You land on the new patient's record.
- **The patient is pushed to the national platform immediately**, if the integration is configured, inside a short budget. If the platform is slow or down, the record is queued instead and the save still succeeds — see §7.

DELETING

Delete on a patient record asks for confirmation first, and says plainly what goes with it: the patient's assessments and lab results are removed too. It cannot be undone, and it does not remove anything already sent to the national platform — that server does not allow deletes.

Sending a DSMA-8 assessment

The questionnaire is the part of the product the *patient* touches. They have no account, so the link itself is the authorisation — which is what makes the rules around it strict.

3.1 Press *Send assessment* on a patient record

- A token is generated — **32 bytes of cryptographic randomness**, never derived from anything about the patient.
- Only its **SHA-256 hash is stored**. The raw token exists in memory for one request and in the patient's inbox. A copy of the database yields no working links.
- It **expires after seven days** and is **single-use**.
- The email is sent, and the link is **also shown to you once**, with a **Copy link** button.

3.2 Why the link is shown to you as well

Because email delivery is not guaranteed and the clinician should not be stuck when it fails. With no email provider configured the app says so and gives you the link; if the provider refuses the recipient, it says that too and gives you the link. Either way the assessment exists and is deliverable.

In this demo that is the normal path, not the fallback: the transactional email provider's free tier only delivers to the account owner's own address, and every patient in the register has a fabricated one. **Expect to copy the link**. The brief explicitly allows it.

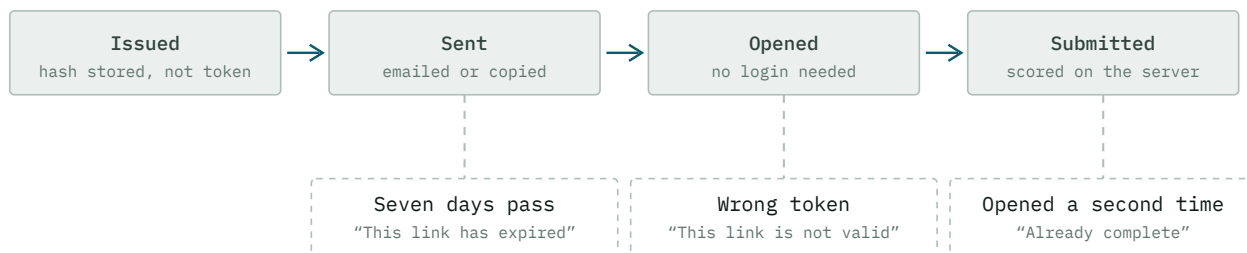
3.3 What the patient sees

A single public page: eight statements about the last two weeks, each answered *Never*, *A few days*, *More than half the days* or *Nearly every day*. All eight are required. The page greets them by first name, and says plainly that it will never ask for a password — because a health link asking for credentials is what a phishing page looks like.

3.4 What happens on submit

- The answers are validated and the total is **recomputed on the server**. A score arriving from the browser is ignored entirely — the number becomes a clinical record, so it is never taken on trust.
- The total, 0–24, is mapped to a risk band.
- The assessment is marked complete, and the link stops working.
- The score appears immediately on the patient's chart, on their row in the register, and in the clinic-wide distribution.

THE LIFE OF ONE LINK



No dead end names the patient, or says whether the link ever existed.

Every dead end is a **plain, identically-shaped message**. Distinguishing “never existed” from “belongs to someone else” would turn a 404 into a way of confirming that a given person is a patient here.

The score, and what each band means

Eight items, each 0–3, summed. Higher is worse. The bands and their guidance come from the supplied questionnaire definition, not from us.

SCORE	BAND	GUIDANCE
0–6	■ Low risk	Routine follow-up.
7–12	■ Moderate risk	Review at next appointment.
13–18	■ High risk	Clinician should contact the patient.
19–24	■ Very high risk	Prompt clinical review recommended.

WHY EVERY BAND IS WRITTEN OUT, NEVER COLOUR ALONE

The four band colours are a continuous green→yellow→orange→red ramp, so neighbouring bands are always close. Measured under deuteranopia, **moderate and high sit at ΔE 0.4** — indistinguishable. Every band therefore carries its written label wherever it appears, and no two adjacent bands are ever drawn as touching segments of one bar.

Importing lab results from a CSV

This is the screen most worth understanding, because it is the one that has to cope with a file somebody else produced. Anyone can accept a well-formed CSV. What the brief actually asks for is that a *messy* file explains itself.

Pressing *Download template*

The link on the import card downloads `lab-results-template.csv` straight from the site — no dialog, no server round trip, nothing is recorded. It is a real file with a header row and three example rows, and it is the shape the importer expects:

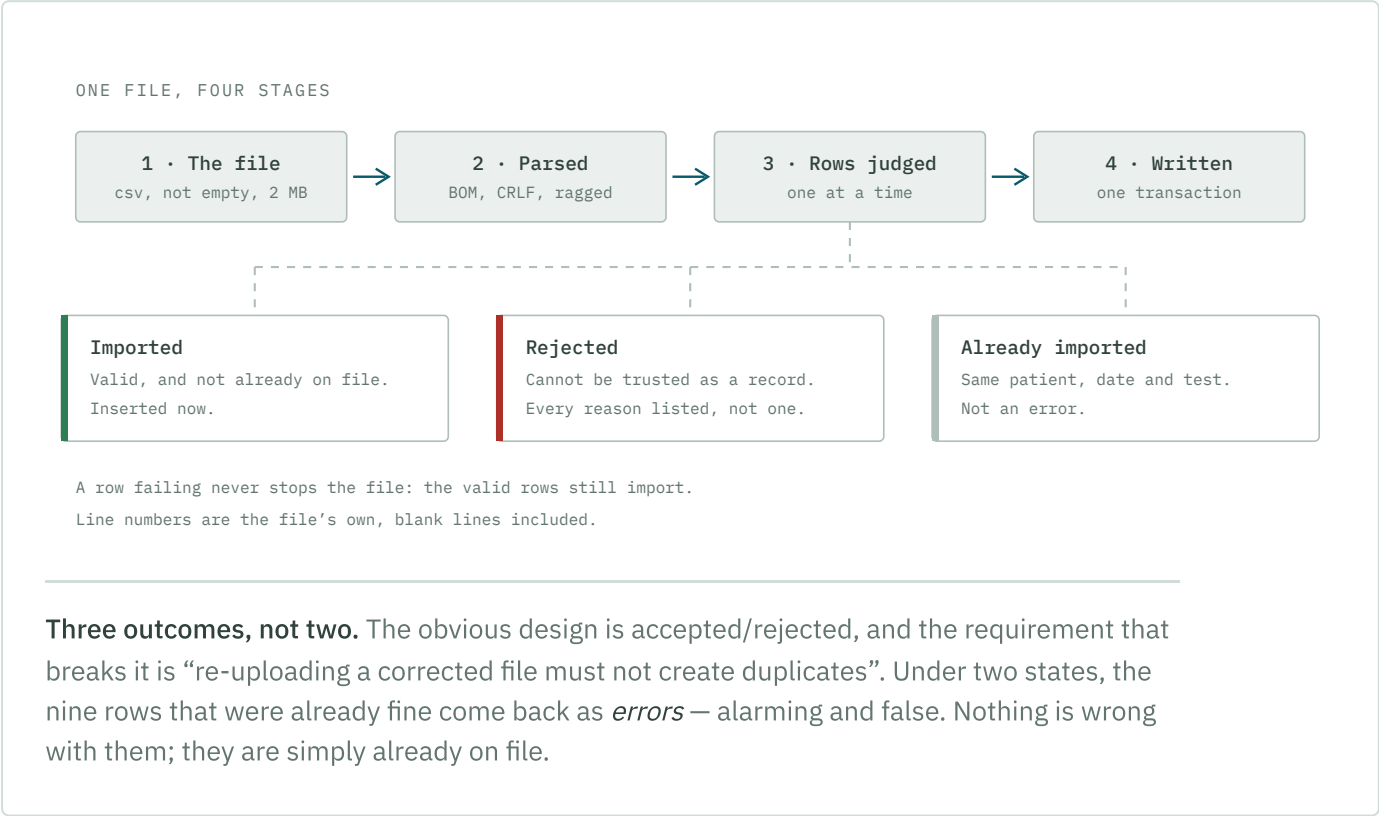
COLUMN	REQUIRED	WHAT IT MUST CONTAIN
<code>mrn</code>	Yes	The MRN of a patient who already exists in the register. Matching ignores case.
<code>collected_date</code>	Yes	<code>YYYY-MM-DD</code> . A real date, not in the future.
<code>test_code</code>	Yes	One of <code>GLU-F</code> , <code>HBA1C</code> , <code>SBP</code> .
<code>value</code>	Yes	Digits and an optional decimal point. Nothing else.
<code>test_name</code>	No	Cosmetic. A mismatch is a note on the row, not a rejection.
<code>unit</code>	No	Checked against the expected unit. A mismatch is flagged and the value is stored exactly as reported .
<code>ref_low</code> , <code>ref_high</code>	No	Chart furniture. Malformed or inverted, and the standard range is used instead with a note.

Three tests are supported, because those are the three the brief's own sample file uses:

CODE	TEST	UNIT	NORMAL RANGE
GLU-F	Fasting Glucose	mg/dL	70 - 99
HBA1C	Hemoglobin A1c	%	4.0 - 5.6
SBP	Systolic Blood Pressure	mmHg	90 - 120

Pressing *Import*

Choose a file, press Import, and it goes through four stages. A failure at any stage is reported in the language of the file you supplied — a line number and a reason — never as a stack trace or a bare count.



What gets a row rejected

Every problem with a row is collected, not just the first. A clinician fixing a file wants all of its faults in one pass rather than one per upload.

FIELD	REJECTED WHEN
MRN	Missing, or no patient in the register has it. The message says to add the patient first or correct the MRN.
Collected date	Missing; not YYYY-MM-DD ; not a real date (2026-02-30); or in the future.
Test code	Missing, or not one of the three supported codes — which the message lists.
Value	Missing; not a plain number; negative; or implausibly large. 7,1 , <5 and 1e3 are all refused rather than guessed at — a decimal comma read as 7 , or a censored result invented as a number, is worse than a rejection.

What only earns a note

An error means the row cannot be trusted as a clinical record. A warning means something was cosmetically off but the measurement is intact — so it imports, and the oddity is reported on the row.

- **A test name that does not match the code.** Stored under the catalogue name, with a note.
- **A unit that does not match the test.** The value is stored **exactly as reported** and flagged — never relabelled. Rewriting a glucose of 5.8 mmol/L as 5.8 mg/dL would turn a normal result into a fatal-looking one while appearing to be a tidy-up. Case and spacing differences normalise silently.
- **A malformed or inverted reference range.** The standard range is used instead, with a note. A reference range is chart furniture; it is never worth rejecting a measurement over.

What gets a row skipped

- **It duplicates an earlier line in the same file** — same patient, date and test. The first wins, and the message names the line it duplicates so you can go and fix the file.
- **It is already on file with the same value.** Nothing to do.
- **It is already on file with a *different* value.** The stored result is **left unchanged** and the message tells you what is on file. Silently overwriting a stored clinical result would make the import non-idempotent — upload the same file twice, get different data — and is the wrong default regardless. Saying so plainly lets the clinician decide.

The report

The result is a per-row table, not a count. Three totals across the top — **Imported, Rejected, Already imported** — then filter chips and one row per line of your file, showing the line number, what was in it, the outcome and the reason.

- If anything was rejected, the table **opens on the rejected rows**. That is what you came to see.
- If every row was already on file, it says so as information, not as a failure.
- If the integration is on, it tells you how many results are now **queued for the national platform**, with a link to send them. They are deliberately not pushed during the import, so a slow platform can never delay this report.

TRY IT WITH THE DELIBERATELY UGLY FILE

`.docs/lab-results-ugly.csv` is 29 rows built to break things: a UTF-8 BOM, CRLF endings, blank lines mid-file, one short row, one long row, and fourteen distinct ways of being invalid. Expected outcome — **11 imported, 14 rejected, 4 already on file** — is written down in `.docs/05-ugly-csv-expected-outcomes.md` before you run it. Import it and the totals should match exactly.

Reading one patient

Open any row in the register. This is the screen for an appointment: who they are at the top and pinned there while you scroll, then the trend, then the underlying records.

5.1 The identity banner

Name, MRN, date of birth, sex, contact details and their current risk band — sticky, so the record you are reading is never in doubt as you scroll. **Send assessment, Edit** and **Delete** live here.

5.2 The trajectory summary

A panel with one **Summarise** button, sitting directly under the banner. It writes a few sentences about where this patient is heading, from their recorded figures and nothing else. It is Tier 3, and [§9](#) is the whole account of how it is kept honest — including the two states where it declines to say anything.

5.3 Four charts, side by side

DSMA-8 score history plus one chart per lab measure — never two measures on one pair of axes. A dual axis lets you imply any correlation you like by choosing the scales, so there isn't one.

- The x-axis is **real time**, not the order rows happened to be stored, so a gap in the record looks like a gap.
- The score chart is **fixed to 0–24**. An auto-scaled axis would rescale between visits and make a two-point improvement look like a collapse.
- A patient with a **single reading** still shows a visible marker, with a week padded either side — every newly created patient hits this.
- Each empty chart says what will appear there and how to make it appear.

5.4 Two tables, abreast

Assessments: when each was sent, its status, when it was completed, the score and the band. **Lab results:** every stored result, newest first, with its value, unit, reference range and where it came from. The tables are the charts in full — **no value is reachable only by hovering**, which matters for anyone reading this on a touchscreen or with a keyboard.

Reading the whole clinic

Clinic is four figures, one distribution and a list of recent imports.

FIGURE	EXACTLY WHAT IT COUNTS
Patients	Everyone in the register, with how many have completed an assessment and how many have not.
Assessment completion	Completed ÷ all sent, all time . Shown as  when nothing has been sent, because 0% of nothing is not 0%.
Higher risk	Patients whose latest completed assessment was High or Very high. It is computed from the same segments the chart below draws, and a test asserts the two agree — so the tile and the chart cannot disagree.
Lab results on file	Every stored result, from CSV imports and from the national platform alike.

Patients by risk band

One labelled bar per band — never stacked, never a pie, because comparing four adjacent hues by area is exactly the task the palette cannot support. Each patient is counted **once**, by their most recent completed assessment. Patients who have never completed one appear as their own *No assessment* segment rather than being dropped, so the segments always sum to the size of the register.

Recent lab imports, and the one filter

The date range — 7, 30, 90 days or all time, defaulting to 30 — **scopes the imports card only**. That is deliberate, and it is the sort of thing a dashboard usually gets wrong. Completion rate is an all-time figure and the risk distribution is a snapshot of the register today; narrowing either to “the last 7 days” would not make it a smaller truth, it would make it a false one.

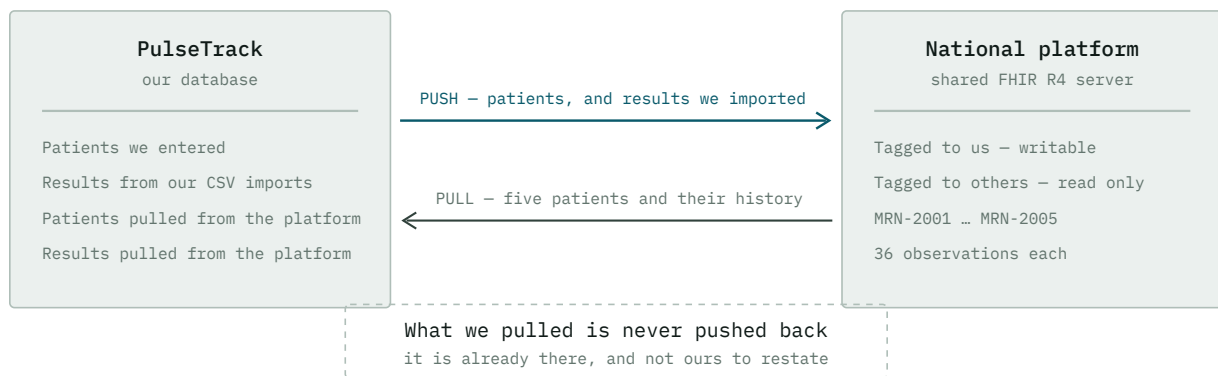
The national platform

A shared FHIR R4 server that every candidate on this challenge writes to. Two directions: our records go out, and twelve months of history for five patients we do not own comes back. One screen — **National platform** — answers the three questions anyone actually has about an external system: is it connected, did our data get there, and what did not.

THE FACT THAT SHAPES EVERYTHING ELSE

Reads on that server are open to everyone; writes are not. Anything we create is tagged as ours, and only we can change it. The five demonstration patients are tagged to the challenge administrator, so we can read their entire history and can never write to them. Five other candidates already hold the MRN we would have used — which is why every write is scoped by our own tag rather than by MRN alone.

TWO DIRECTIONS, ONE RULE



The **Results sent** figure on the page reads *“locally-entered results only”* for exactly this reason: pushing pulled data back would duplicate a record the platform already holds, under our tag, where it would then look like our observation.

What the page shows before you press anything

- **Connection** — the server address and our organisation tag, and whether the integration is configured at all. If it is not, the page says which environment variables are missing and states that **the rest of PulseTrack works without them**.

- **Six figures** — patients linked, results sent, queued, last sent, patients imported, results imported.
- All of it is read from **our own rows**, never from the platform. So the page renders in full when the platform is unreachable — which is precisely when someone opens it.

Pressing *Push*

- A batch of **20 records** per request, repeating until the queue is empty. Each batch is bounded so it finishes well inside the serverless function's budget rather than being cut off by it.
- A patient referenced by a queued result is **linked first**, before any result is sent. Without that ordering, two results for the same unlinked patient would each create that patient, on a server where a duplicate cannot be deleted.
- Writes are **conditional**: creating the same patient twice returns the existing record rather than a second copy. Re-running a push that already succeeded writes nothing.
- **Email and phone are never sent**. Reads are open to every other organisation on that server, and a demonstration integration has no business publishing contact details.

Pressing *Import*

- Five MRNs, **one request each**, so a slow platform cannot take the whole import over the time limit — and so the row for MRN-2003 can say what happened to MRN-2003 rather than being lost in one aggregate count.
- The MRNs are an **allow-list**, not a text box. An authenticated session cannot be turned into a way of reading arbitrary records off a shared server.
- Each patient's history is **36 observations across two pages**. Paging is followed to the end; the loop stops when there is no next page, never on a total the server may not send.
- Imported results land in the same tables as local data, so they appear on the same charts, in the same tables, and in the same clinic figures.
- Run it twice and the second run writes nothing: rows are compared before being written, so an unchanged re-import is idempotent *and* cheap.

When a result already exists locally

The same rule as the CSV importer, for the same reason: the link to the platform's record is attached, and **the stored value is never overwritten**. A disagreement between what we hold and what the platform holds is counted and shown, not silently resolved in either direction.

Records the platform would not accept

A table at the foot of the page, never a silent retry. It distinguishes **Refused** — the platform will not let us write that record, which is a fact about ownership and will not improve on a retry — from **Failed**, a transient fault worth trying again. Transient failures are retried automatically with a backoff; refusals are surfaced, with whatever the platform actually said.

TIER 3

The trajectory summary

One button, on a screen the clinician is already reading. It turns a patient's lab trends and questionnaire scores into a few sentences of plain prose. The design question underneath it is the one the brief asks: how do you put a language model near clinical data without it inventing any of that data. The answer here is that **the model is never allowed to calculate anything** — it is handed the figures and asked only to narrate them.

THE FACT THAT SHAPES EVERYTHING ELSE

Every number in the summary is computed in TypeScript *before* the model is called, and checked again *after* it replies. The prompt does tell it to use no figures but the ones supplied — but that instruction alone is unfalsifiable and fails silently, so it is not what the guarantee rests on. What it rests on is the check afterwards: every number is pulled back out of the prose and matched against the source. **A summary containing a figure that is not in this patient's records is discarded and never shown.**

The five things the panel can say

WHAT YOU SEE	WHEN, AND WHY IT IS WORDED THAT WAY RATHER THAN AS AN ERROR
A summary , with the figures printed beside it	The normal path. The prose and its source sit together, so the summary is read <i>against</i> the numbers rather than instead of them.
Not enough data to summarise	Fewer than two recorded data points — lab results, completed assessments, or one of each. Narrating a single reading is one sentence restating that reading, which is worse than an empty state saying so plainly. The threshold is deliberately low, because the cost of refusing is a clinician seeing nothing at all.
Summary withheld	The generated note referred to a figure that does not appear in this patient's records, so it was discarded rather than shown. The recorded figures are still displayed. This state is the safeguard working , not a fault, and it is worded that way.
The summary could not be generated	The provider failed or timed out. The message is our own classification of the failure, never the provider's response body — that body can carry the prompt back, and the prompt carries patient data.
Not configured on this deployment	<code>AI_PROVIDER</code> and <code>AI_API_KEY</code> are unset. Everything else in the app runs untouched; Tier 3 is the only thing that needs them.

What the model is given

A compact payload, and no database access: age and sex, each lab measure with its readings and direction of travel, the DSMA-8 scores and how many were completed, and how many days have passed since the most recent data of any kind. Three details in that payload exist purely to remove a way of being wrong.

- The **scale is supplied** — 0–24 — so the model never has to recall it, and cannot describe a score of 19 as though the ceiling were 40.
- The **direction is supplied**: on DSMA-8 a **higher score is worse**, because every item is negatively worded. It is the single easiest thing for a model to get backwards, and getting it backwards inverts the entire summary. It is stated in the payload, stated again in the prompt, and a test fails if the wording is removed.
- Every figure is **rounded once**, in the code that builds the payload. The number the model is shown and the number the check will accept are then the same number by construction — round it later and the model faithfully echoes `6.8` while the check holds `6.799999999999999` and rejects a summary that was correct.

What the check does, and what it cannot do

It catches **invented and mistyped numbers**, which is the failure mode that matters here because every clinical statement in this summary is numeric. Dates are stripped out before anything is counted — a date is not a clinical figure, and admitting every year and day into the allowed set would let a fabricated score of `14` pass because some reading happened on the 14th. Names containing digits are stripped for the same reason, so `HbA1c` and `DSMA-8` are not read as two invented figures.

It does **not** catch a fluent, entirely non-numeric wrong claim — “the patient is managing well” written over a deteriorating trend. Nothing mechanical would. That is exactly why the computed figures are rendered beside the prose rather than behind it, and why this panel is a note to a clinician who can see the charts on the same screen, never a decision.

WHAT IT IS NOT

It is not a diagnosis, a triage tool or a recommendation. It carries a disclaimer and names the provider and model that produced it, and it is generated only when somebody presses the button — never on page load, never in a batch. A clinician who ignores the panel entirely loses nothing: everything it says is already on the screen, in the charts and the tables.

THE RULES

Eight things the app will not do, whatever you click

It will not trust a score from a browser

Assessment totals are always recomputed on the server from the stored answers. A total posted by a client is discarded before it is read.

It will not overwrite a stored clinical result

Neither a re-uploaded CSV row nor an imported platform observation may change a value already on file. The conflict is reported and the clinician decides.

It will not create a duplicate from a repeated import

A result is identified by patient, date and test. Re-uploading the same file, or re-running the same import, writes nothing at all.

It will not reject a whole file for one bad row

Rows are judged individually. Eleven valid rows import even when fourteen others do not.

It will not tell an anonymous visitor whether a record exists

An invalid assessment link, an expired one and one belonging to somebody else all produce the same shape of message. A 404 that distinguishes them is a way to enumerate patients.

It will not log patient data

Server logs carry ids and counts. No names, no email addresses, no MRNs, no answers, no tokens.

It will not let a hidden button be the security boundary

Every mutation re-authorises on the server. The edge check and the missing button are conveniences.

It will not publish contact details to the shared platform

Email and phone stay in our database. Everything else on that server is readable by every other organisation using it.

SCOPE

What it deliberately does not do

Each of these is a decision, not an omission. Naming them is cheaper than letting a reader discover them and wonder.

- **No patient accounts.** Patients hold a single-use link and nothing else.
- **No roles or permissions.** One clinician role; there is no admin tier to get wrong.
- **No editing of a stored lab result.** Delete and re-import instead — a mutable clinical value needs an audit trail this does not have.
- **No reminders or scheduling.** Assessments are sent when a clinician sends them.
- **No background jobs.** The push queue drains when someone presses Push; a demonstration deployment has no worker.
- **No dark mode in the app.** One well-executed light theme, deliberately — clinical software is read in bright rooms and printed.
- **No compliance claim.** The practices here are good practice. Good practice is not HIPAA or GDPR compliance, and this document will not pretend otherwise.
- **No writes back to another organisation's records** on the national platform, which that server would refuse anyway.

Where each requirement is satisfied

Tier 1 — the brief's six areas

REQUIREMENT	WHERE YOU SEE IT
Authentication	<code>/login</code> , and the three-layer check behind every other screen. §2
Patient management	Patients — register, search, add, edit, delete. §3
Email questionnaire flow	Send assessment on a patient, then the public questionnaire page. §4
CSV lab upload	Lab import , and the per-row report it produces. §5
Dashboards	Clinic , and the four charts on each patient. §6, §7
Documentation	The README, and this guide.

Tier 2 — the brief's four requirements

REQUIREMENT	WHERE YOU SEE IT
Push patients and lab results	National platform → Push , plus the automatic push when a patient is saved. §8
Pull the seeded patients and their history	National platform → Import — five patients, 36 observations each. §8
Authentication, failures, retries, no double-import	The connection card, the failures table, and the fact that a second run writes nothing. §8
An integration diagram	The README, which carries the integration diagram and the sequence of a sync.

Tier 3 – the brief’s four questions

THE QUESTION THE BRIEF ASKS	THE ANSWER, AND WHERE YOU SEE IT
Is it grounded in the real data?	Every number is computed in TypeScript first; the model only narrates. The figures are printed beside the prose. §9
Is hallucination risk handled?	Three ways rather than one: the prompt, a mechanical check of the output that discards any summary carrying an unsupported figure, and the source figures shown alongside. §9
Is the prompt design thoughtful?	Narrator, not analyst. The scale and the score direction are supplied rather than recalled, and the direction is stated twice and covered by a test. §9
Is it actually useful, or a demo?	One button on a page the clinician already reads, and it refuses rather than pads when there is too little to say. §9

IF YOU HAVE TEN MINUTES AND WANT TO SEE EVERYTHING WORK

Sign in → **Clinic** to see the register at a glance → open a patient and read the four charts → **Send assessment**, copy the link, open it in a private window and complete it, then reload the patient → **Lab import**, download the template, then import `.docs/lab-results-ugly.csv` and check the totals against `05-ugly-csv-expected-outcomes.md` → **National platform**, press **Import**, then press it again and watch the second run write nothing → back on a patient, press **Summarise** and read the note against the figures printed beside it.